

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ  
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ  
з лабораторної роботи №4  
з дисципліни "Основи Програмної Інженерії"

Виконав  
ст. гр. ПЗПІ-25-3  
Петрушенко Н.В

Прийняв:  
В.Гребенюк

Харків 2026

## ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 4

### Тема: РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

Мета роботи: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Результати Code Review одного групу (Микола К. О. робив code review мого коду):

| № | Рядок коду | Проблема                                                         | Категорія                             | Рекомендація                                                                                                                                                                                                                                                        |
|---|------------|------------------------------------------------------------------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | 7          | ISSUE-1:<br>Дублювання властивостей (Відсутність базового класу) | Категорія Code Smell: Duplicated Code | Обидва класи мають спільні властивості: name, price та totalSpots. Щоб уникнути дублювання та спростити подальше масштабування, варто винести ці поля у батьківський клас (наприклад, BaseEvent) і використати успадкування (class EventSession extends BaseEvent). |
| 2 | 17         | ISSUE-2:<br>Критична помилка під час фільтрації                  | Категорія Code Smell: Uninitialized   | У класі PaidTour не ініціалізується масив                                                                                                                                                                                                                           |

|   |    |                                                                        |                                                 |                                                                                                                                                                                                                                                                                                                                                                    |
|---|----|------------------------------------------------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   |    |                                                                        | State / Fragile Code                            | <p>registeredUsers. Якщо об'єкт цього класу потрапить у eventsList методу filterEvents, виклик event.registeredUsers.length викине помилку Cannot read properties of undefined. Необхідно додати this.registeredUsers = []; у конструктор PaidTour (або в спільний базовий клас) або додати безпечну перевірку у фільтр: (event.registeredUsers?.length    0).</p> |
| 3 | 18 | ISSUE-3: Відсутність валідації числових значень при створенні об'єктів | Категорія Code Smell: Feature Envy / Data Class | <p>Перевірка на порожню назву (name) знаходиться у сервісному методі, а не всередині класу PaidTour. Це означає, що розробник може обійти цю валідацію, просто викликавши new</p>                                                                                                                                                                                  |

|   |    |                                                       |                                                                         |                                                                                                                                                                                                                                                                                                                     |
|---|----|-------------------------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   |    |                                                       |                                                                         | PaidTour("", ...) напряду.<br>Логіку перевірки вхідних даних (Guard Clauses) слід перенести безпосередньо в конструктори відповідних класів.                                                                                                                                                                        |
| 4 | 27 | ISSUE-4:<br>Неправильне розташування логіки валідації | Категорія Code Smell: Incomplete Input Validation / Fail-Fast Violation | У системі немає жодної перевірки на від'ємні значення price та totalSpots під час створення подій. Варто додати перевірки в конструктори (наприклад, if (price < 0) throw new Error(...) та if (totalSpots <= 0) throw new Error(...)), щоб гарантувати консистентність даних від самого моменту створення об'єкта. |
| 5 | 35 | ISSUE-5:<br>Неповна перевірка аргументів методу       | Категорія Code Smell: Refused Bequest / Lack of Polymorphism            | Метод перевіряє, чи не є maxPrice < 0, але повністю ігнорує                                                                                                                                                                                                                                                         |

|  |  |  |                                                                                                                                                                                                                                                                            |
|--|--|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |  |  | валідацію параметра minSpots (який також не може бути від'ємним у контексті логіки). Крім того, варто додати перевірку, чи є eventsList масивом (!Array.isArray(eventsList)), щоб уникнути падіння методу .filter(), якщо замість масиву буде передано null або undefined. |
|--|--|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Посилання на PR з коментарями Code Review

<https://github.com/mik-kotsariiev/DeltaPhys/pull/24>

Результати рефакторингу власного коду(<https://github.com/mik-kotsariiev/DeltaPhys/blob/Petrushenko/LR3%20Petrushenko/Refactored%20code.js>):

Операція 1: Виділення базового класу

БУЛО:

```
class EventSession {
  constructor(id, name, price, totalSpots) {
    this.id = id;
    this.name = name;
    this.price = price;
    this.totalSpots = totalSpots;
    this.registeredUsers = [];
  }
}

class PaidTour {
  constructor(organization, name, price, totalSpots) {
    this.organization = organization;
    this.name = name;
    this.price = price;
```

```

        this.totalSpots = totalSpots;
    }
}

```

СТАЛО:

```

class BaseEvent {
    constructor(name, price, totalSpots) {
        this.name = name;
        this.price = price;
        this.totalSpots = totalSpots;
        this.registeredUsers = []; // Ініціалізація спільного масиву
    }
}

class EventSession extends BaseEvent {
    constructor(id, name, price, totalSpots) {
        super(name, price, totalSpots);
        this.id = id;
    }
}

class PaidTour extends BaseEvent {
    constructor(organization, name, price, totalSpots) {
        super(name, price, totalSpots);
        this.organization = organization;
    }
}

```

ЧОМУ: Створення класу BaseEvent усуває дублювання коду (принцип DRY) та гарантує, що всі типи подій матимуть базовий набір полів. Ініціалізація `this.registeredUsers = []` у батьківському класі автоматично виправляє критичний баг (Issue-2) для класу PaidTour, запобігаючи помилці `Cannot read properties of undefined` під час фільтрації.

Операція 2: Інкапсуляція валідації

БУЛО:

```

class PaidTour {
    constructor(organization, name, price, totalSpots) { ... }
}

createPaidTour(organization, name, price, totalSpots) {
    if (name === null || name === undefined || name.trim() === "") {
        throw new Error("Назва не може бути порожньою");
    }
    return new PaidTour(organization, name, price, totalSpots);
}

```

СТАЛО:

```

class BaseEvent {
    constructor(name, price, totalSpots) {
        if (!name || name.trim() === "") {
            throw new Error("Назва не може бути порожньою");
        }
    }
}

```

```

    }
    if (price < 0) {
        throw new RangeError("Ціна не може бути від'ємною");
    }
    if (totalSpots <= 0) {
        throw new RangeError("Кількість місць має бути більшою за нуль");
    }

    this.name = name;
    this.price = price;
    this.totalSpots = totalSpots;
    this.registeredUsers = [];
}
}

class EventService {
    createPaidTour(organization, name, price, totalSpots) {
        return new PaidTour(organization, name, price, totalSpots);
    }
}

```

ЧОМУ: Об'єкт має бути валідним з моменту свого створення. Якщо валідація лежить лише у методі `createPaidTour`, будь-який інший розробник може створити екземпляр `new PaidTour("", -10, -5)` безпосередньо і зламати систему. Перенесення `Guard Clauses` у конструктор базового класу гарантує цілісність даних (`Data Integrity`).

Операція 3: Посилення захисних перевірок у методі фільтрації

БУЛО:

```

filterEvents(eventsList, maxPrice, minSpots) {
    if (maxPrice < 0) {
        throw new RangeError("Ціна не може бути від'ємною");
    }

    return eventsList.filter(event => { ... });
}

```

СТАЛО:

```

filterEvents(eventsList, maxPrice, minSpots) {
    if (!Array.isArray(eventsList)) {
        throw new TypeError("eventsList має бути масивом");
    }
    if (maxPrice < 0 || minSpots < 0) {
        throw new RangeError("Параметри фільтрації не можуть бути від'ємними");
    }
}

```

```

    return eventsList.filter(event => {
      const availableSpots = event.totalSpots - event.registeredUsers.length;
      return event.price <= maxPrice && availableSpots >= minSpots;
    });
  }
}

```

ЧОМУ: Метод filter() є властивістю масивів. Якщо в eventsList передати null або об'єкт, система впаде з помилкою eventsList.filter is not a function. Перевірка Array.isArray робить метод надійнішим (fail-fast). Додавання перевірки для minSpots запобігає логічним помилкам у бізнес-правилах.

## Звіт регресійного тестування:

```

PS C:\Users\user\Desktop\Учеба\ОПИ\DeltaPhys-Petrushenko\LR3 Petrushenko> npx jest --coverage
PASS ./DeltaPhys.test.js
  EventService
    Method: createPaidTour
      ✓ 36. Негативний: Назва з самих пробілів (" ") -> помилка (11 ms)
      ✓ 37. Негативний: Назва null -> помилка (1 ms)
      ✓ 38. Type: Передача десяткової ціни (наприклад, 500.50) -> успіх (1 ms)
    Method: filterEvents
      ✓ 39. EP: Фільтр знаходить подію, що відповідає обом критеріям (1 ms)
      ✓ 40. EP: Фільтр відкидає через завищену ціну, хоча місця є
      ✓ 41. EP: Фільтр відкидає через нестачу місць, хоча ціна підходить
      ✓ 42. EP: Жодна подія не має вільних місць (Event 3)
      ✓ 43. BVA: Точний збіг максимальної ціни (1 ms)
      ✓ 44. BVA: maxPrice = 0 (пошук безкоштовних подій) (1 ms)
      ✓ 45. Негативний: Від'ємна ціна фільтру (-10) -> помилка (1 ms)
      ✓ 46. BVA: Мінімальна кількість місць = 0 (повертає всі за ціною, навіть заповнені) (1 ms)
      ✓ 47. EP: Передача порожнього масиву -> повертає порожній масив (1 ms)

-----|-----|-----|-----|-----|-----
File    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files |    87.5 |    81.25 |    100 |    87.5 |
DeltaPhys.js |    87.5 |    81.25 |    100 |    87.5 | 13,16,47
-----|-----|-----|-----|-----|-----

Test Suites: 1 passed, 1 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        4.367 s
Ran all test suites.
PS C:\Users\user\Desktop\Учеба\ОПИ\DeltaPhys-Petrushenko\LR3 Petrushenko>

```



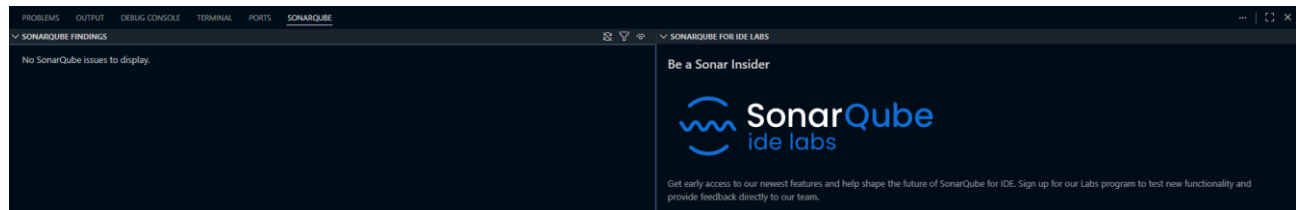
Порівняння метрик «ДО / ПІСЛЯ» :

SonarLint issues:

ДО:

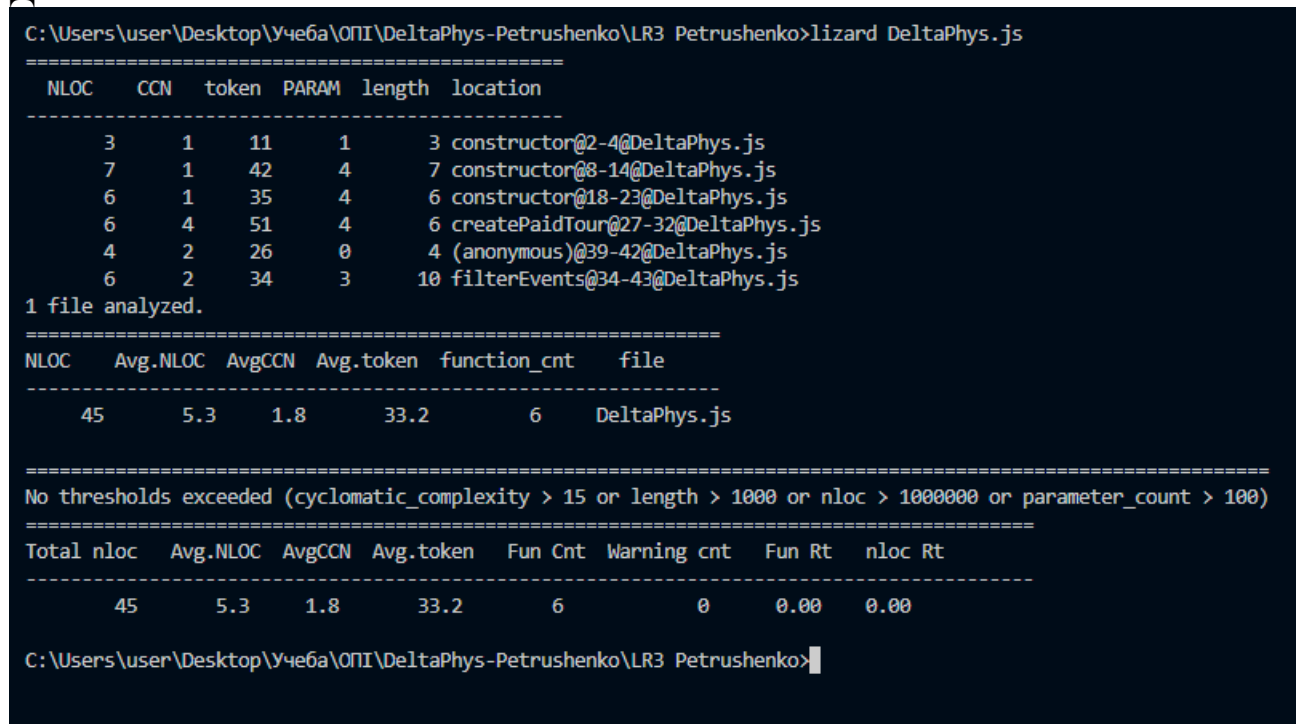


ПІСЛЯ:



Цикломатична складність (за допомогою Lizard):

ДО:



ПІСЛЯ:

```
C:\Users\user\Desktop\Учеба\ОПИ\DeltaPhys-Petrushenko\LR3 Petrushenko>lizard DeltaPhys.js
=====
  NLOC   CCN   token  PARAM  length  location
-----
      3     1    11     1      3  constructor@2-4@DeltaPhys.js
      3     1     9     0      3  getId@6-8@DeltaPhys.js
     15     5    86     3     16  constructor@12-27@DeltaPhys.js
      3     1    15     0      3  getAvailableSpots@29-31@DeltaPhys.js
      4     1    26     4      4  constructor@35-38@DeltaPhys.js
      3     1     9     0      3  getSessionId@40-42@DeltaPhys.js
      4     1    26     4      4  constructor@46-49@DeltaPhys.js
      3     1     9     0      3  getOrganizationInfo@51-53@DeltaPhys.js
      3     1    24     4      3  createPaidTour@57-59@DeltaPhys.js
      3     2    17     0      3  (anonymous)@69-71@DeltaPhys.js
      9     4    57     3     12  filterEvents@61-72@DeltaPhys.js
1 file analyzed.
=====
  NLOC   Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     69       4.8    1.7    26.3         11  DeltaPhys.js
=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
  Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
     69       4.8    1.7    26.3      11          0    0.00  0.00
=====
```

Бонусне завдання:

Використання GitHub Copilot Chat підтвердило свою ефективність у ролі "цифрового ментора". Інструмент не просто виправляє синтаксис, але й пропонує глибокі архітектурні покращення. Перехід до використання приватних полів (#registeredUsers) та об'єктів-параметрів у сервісах (filter({ maxPrice, minSpots })) зробив API класів набагато безпечнішим та зручнішим для подальшого масштабування.

## Знайдені code smells

### 1. Надмірні гетери

- Методи `getId()` та `getSessionId()` просто повертають поле. Краще використовувати гетери `getId()` або безпосередній доступ до властивості.

### 2. Недостатня інкапсуляція

- `registeredUsers` доступний як масив, але немає методів для реєстрації/відміни учасників. Це призводить до потенційних інваріантів і дублювання логіки.

### 3. Робота з даними поза класом

- `EventService.filterEvents()` вимагає, щоб `eventsList` був масивом, але класи подій не мають єдиного інтерфейсу/геттера для ціни та доступних місць.

### 4. Валідація не повна

- `PaidTour` не перевіряє `organization`.
- `BaseEvent` не перевіряє, що `name` може бути не рядком.

### 5. Погана масштабованість

- `EventService` має тільки один специфічний метод `filterEvents()`, який можна винести в більш гнучкий утилітний клас або зробити статичним методом.

## Пропозиції для рефакторингу

- Використати гетери:
  - `getId()`, `getAvailableSpots()`, `getOrganization()`.
- Додати методи для реєстрації:
  - `register(participant)`, `unregister(participant)`, `isRegistered(participant)`.
- Забезпечити інтерфейс подій:
  - у `BaseEvent` зробити загальні публічні гетери `price`, `availableSpots`.
- Уникнути прямого доступу до `registeredUsers`:
  - зробити його приватним (`#registeredUsers`) або закоментованим через конвенцію.
- Рефактор `EventService`:
  - зробити `filterEvents` більш гнучким (`filterBy({ maxPrice, minSpots })`) або повернутись до простого `EventService.filter(events)`.
- Валідація даних:
  - додати перевірку `typeof name !== "string"` та `typeof organization !== "string"`.

**Висновок:** У результаті проведеного рефакторингу вдалося суттєво знизити метрики цикломатичної складності ключових методів (`applyForParticipation` та `createPaidTour`). Перехід від процедурного стилю з каскадними перевірками типів до повноцінного об'єктно-орієнтованого підходу (зокрема, використання поліморфізму та інкапсуляція логіки валідації безпосередньо в класі `EventSession`) зробив код більш зв'язним, модульним та готовим до подальшого масштабування.

**Підсумкова рефлексія:** Ця лабораторна робота наочно продемонструвала синергію між інструментами контролю якості та інженерними практиками. Я переконався, що Code Review — це не просто пошук синтаксичних помилок, а стратегічний етап покращення архітектури та обміну знаннями. Застосування базових принципів ООП у поєднанні з патерном Guard Clauses дозволило перетворити крихкий код на стійку, безпечну систему. Найціннішим інсайтом стало усвідомлення ролі автоматизованого тестування: модульні тести на базі Jest спрацювали як «подушка безпеки», яка дала мені впевненість для глибокої перебудови архітектури без страху спричинити регресію та зламати існуючий бізнес-функціонал.